Alliance University · KubePilot Autoscaling Framework · Windows CMD & PowerShell Reference

Project: KubePilot — Predictive & Reactive Autoscaling in Kubernetes (Project 357)
Environment: Minikube (Docker driver) on Windows 10/11
Target Workload: Namespace `autoscaling-demo` | Deployment `sample-web-app` (Port 8080)
Monitoring: Namespace `monitoring` | Service `prometheus-server` (Port 9090)

---

1. Directory Setup

Always execute your terminal commands from the repository root:

PowerShell

```
cd C:\Users\SRIRAM\design_project
```

Windows CMD

```
cd /d C:\Users\SRIRAM\design_project
```

---

2. Cluster Health & Minikube Verification

Confirm that Minikube is active and that your cluster nodes and namespaces are ready:

PowerShell & CMD

```
minikube status
kubectl config current-context
kubectl get nodes -o wide
kubectl get namespaces
```

Check Workload and Prometheus Resources:

```
kubectl get pods,svc,deploy -n autoscaling-demo -o wide
kubectl get pods,svc,deploy -n monitoring -o wide
```

Expected Status:

- In `autoscaling-demo`: Deployment `sample-web-app` has `2/2` ready pods in `Running` state and Service `sample-web-app` exposes port `8080/TCP`.
- In `monitoring`: Deployment `prometheus-server` has `1/1` ready pod in `Running` state and Service `prometheus-server` exposes port `9090/TCP`.

---

3. Port-Forwarding (Open in Dedicated Terminals)

Because Minikube operates within an isolated container network, you must forward ports to access services from your Windows host (`localhost`).

Terminal 1 — Prometheus Service (Port 9090)

Required for the telemetry pipeline and Prometheus UI:

```
kubectl port-forward -n monitoring svc/prometheus-server 9090:9090
```

> *Leave this terminal window running.*

Terminal 2 — Sample Web Workload (Port 8080)

For inspecting application health and application-level metrics:

```
kubectl port-forward -n autoscaling-demo svc/sample-web-app 8080:8080
```

> *Leave this terminal window running.*

---

4. Test Prometheus & Scrape Targets via CMD / PowerShell

Once port 9090 is forwarded, verify Prometheus health and target scraping:

In PowerShell:

```
# 1. Verify Prometheus Ready state
Invoke-RestMethod -Uri "http://localhost:9090/-/ready"

# 2. Verify Prometheus Health
Invoke-RestMethod -Uri "http://localhost:9090/-/healthy"

# 3. Inspect active scrape targets and status
(Invoke-RestMethod -Uri "http://localhost:9090/api/v1/targets").data.activeTargets | Select-Object discove
redLabels, health, lastScrape
```

In Windows CMD:

```
:: 1. Verify Prometheus Ready state
curl -s http://localhost:9090/-/ready

:: 2. Verify Prometheus Health
curl -s http://localhost:9090/-/healthy

:: 3. Inspect active scrape targets
curl -s http://localhost:9090/api/v1/targets
```

---

5. Test Telemetry Pipeline via PromQL Queries

Run the exact PromQL queries configured in `controller/telemetry.py` to confirm metrics are flowing from the cluster to Prometheus:

In PowerShell:

```
# Query 1: Container CPU Usage
Invoke-RestMethod -Uri 'http://localhost:9090/api/v1/query?query=sum(rate(container_cpu_usage_seconds_tota
l{namespace="autoscaling-demo"}[1m]))'

# Query 2: Container Memory Working Set
Invoke-RestMethod -Uri 'http://localhost:9090/api/v1/query?query=sum(container_memory_working_set_bytes{na
mespace="autoscaling-demo"})'

# Query 3: Workload HTTP Requests per Second
Invoke-RestMethod -Uri 'http://localhost:9090/api/v1/query?query=sum(rate(sample_app_requests_total[1m]))'
```

In Windows CMD:

```
:: Query 1: Container CPU Usage
curl -sG "http://localhost:9090/api/v1/query" --data-urlencode "query=sum(rate(container_cpu_usage_seconds
_total{namespace=\"autoscaling-demo\"}[1m]))"

:: Query 2: Container Memory Working Set
curl -sG "http://localhost:9090/api/v1/query" --data-urlencode "query=sum(container_memory_working_set_byt
es{namespace=\"autoscaling-demo\"})"

:: Query 3: Workload HTTP Requests per Second
curl -sG "http://localhost:9090/api/v1/query" --data-urlencode "query=sum(rate(sample_app_requests_total[1
m]))"
```

---

6. Test the Workload Application Directly (Port 8080)

In PowerShell:

```
# Health probe
Invoke-RestMethod -Uri "http://localhost:8080/healthz"

# Application Prometheus metrics endpoint
Invoke-RestMethod -Uri "http://localhost:8080/metrics"
```

In Windows CMD:

```
curl -i http://localhost:8080/healthz
curl -s http://localhost:8080/metrics
```

---

7. Run the KubePilot Application (Dashboard & Controller)

Execute these from `C:\Users\SRIRAM\design_project`:

Run the Web Dashboard:

```
python -m dashboard.app
```

- Web interface URL: **`http://localhost:5055`**
- Features: Live Telemetry, Scaling Policies, Decision Engine, Workloads, and Execution History.

Run the Autonomous PRAH Controller (Separate Terminal):

```
python -m controller.main
```

- Continuously polls Prometheus metrics, checks policies, evaluates cooldown/guardrails, and manages scaling.

---

8. Validate Dashboard Telemetry APIs via Terminal

While `python -m dashboard.app` is running, verify that the backend telemetry pipeline reports connected status:

In PowerShell:

```
# Check Kubernetes & Prometheus integration health
Invoke-RestMethod -Uri "http://localhost:5055/api/health" | ConvertTo-Json -Depth 4

# Check active telemetry sample values
Invoke-RestMethod -Uri "http://localhost:5055/api/telemetry" | ConvertTo-Json -Depth 4

# Check decision engine evaluation status
Invoke-RestMethod -Uri "http://localhost:5055/api/decisions" | ConvertTo-Json -Depth 4

# Check execution history records
Invoke-RestMethod -Uri "http://localhost:5055/api/history" | ConvertTo-Json -Depth 4
```

In Windows CMD:

```
curl -s http://localhost:5055/api/health
curl -s http://localhost:5055/api/telemetry
curl -s http://localhost:5055/api/decisions
curl -s http://localhost:5055/api/history
```

---

9. Automated Test Suite

To run all 86 unit and integration tests across the telemetry provider, decision engine, executor, and dashboard:

```
python -m pytest tests/ -v
```

---

10. Quick Troubleshooting Reference

Symptom	Cause	Solution
`Cannot connect to Prometheus at http://localhost:9090`	Port-forward is not running	Run `kubectl port-forward -n monitoring svc/prometheus-server 9090:9090` in an active terminal.
`No matching series returned by Prometheus`	Pods not yet scraped or uninstrumented	Confirm pods are `Running` via `kubectl get pods -n autoscaling-demo` and generate traffic to `/`.
`Kubernetes connection unavailable`	Minikube stopped or context mismatch	Run `minikube status` and `kubectl config use-context minikube`.
Port 5055 already in use	Previous dashboard instance still running	Run `netstat -ano | findstr :5055` and terminate the orphaned PID using `taskkill /PID <PID> /F`.